

```
///Make A Substring Of A String
//substr(starting index, number of elements)
temp = str.substr(0, 3);

///Insert A New_String Anywhere(Any Index) Inside Another String
str.insert(0, new_string);

///Search For The Index Position Of A Character Or A Substring Inside A String
int index = str.find(s);

///Erase Portion Of A String
//erase(starting index, number of character to erase);
str.erase(0, 4);

///To Swap String
str.swap(src);
///Reverse A String
reverse(str.begin(), str.end());

///Compare string with other string

///Replace A Portion Of A String With Another String
//replace(start index, number of char to replace, another string);
str.replace(i,1,s);

///Removes Last Char Of A String
str.pop_back();

///Add A Char(Not String) In The End Of A String
str.push_back(ch);

return 0;
}

//For strings like MmmMMEeeEEooOooOwwwWWWW
int main(){
    int t,n;
    string s;
    for(cin>>t;cin>>n>>s;)
        cout<<(regex_match(s,regex("[mM]+[eE]+[oO]+[wW]+"))?"YES":"NO")<<endl;
}
```

//Mod

Properties

- The value of $a \bmod m$ is always between 0 and $m - 1$.
- $a \equiv b \pmod{m}$ means that a and b have the same remainder when divided by m . That is $a \bmod m = b \bmod m$.
- $(a + b) \equiv (a \bmod m + b \bmod m) \pmod{m}$.
- $(a - b) \equiv (a \bmod m - b \bmod m) \pmod{m}$.
- $(a \cdot b) \equiv (a \bmod m) \cdot (b \bmod m) \pmod{m}$.
- $a^b \equiv (a \bmod m)^b \pmod{m}$.
- $a \bmod m = b \bmod m \implies (a - b) \bmod m = 0$. That is, if a and b have the same remainder when divided by m , then $a - b$ is divisible by m .
- We can also do divisions but we will learn about it later.

//GCD

`_gcd(a, b)`

//LCM

$$\text{lcm}(a, b) = \frac{a \times b}{\text{gcd}(a, b)}$$

Useful Builtin Functions

- `abs`
- `ceil`, `floor`, `round`, `llround`
- `log`, `log2`, `log10`
- `sqrt`, `cbrt`, `pow`
- `stoi`, `stoll`, `to_string`
- `cerr`, `cout`
- `isalpha`, `isdigit`, `islower`, `isupper`, `tolower`, `toupper`
- `swap`
- `gcd`
- `rand`
- `reverse`
- `sort`, `is_sorted`

- **Sum of first n positive integers:** $1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$. Why?
- **Sum of first n odd positive integers:** $1 + 3 + 5 + \dots + (2n - 1) = n^2$. Why?
- **Sum of first n even positive integers:** $2 + 4 + 6 + \dots + 2n = n(n + 1)$. Why?
- **Sum of first n squares:** $1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$.
- **Sum of first n cubes:** $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4} = \left(\frac{n(n+1)}{2}\right)^2$.

- **Factorial:** $n!$ is the number of ways to arrange n distinct objects in a line. It is denoted by $n!$ notation. It is defined as $n! = 1 \times 2 \times 3 \times \cdots \times n$. Examples: $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$. $0!$ is defined to be 1. Why? Because $n!$ is the number of ways to arrange n distinct objects in a line. There is only one way to arrange zero objects in a line: do nothing. So $0! = 1$.
- **Combinations:** $\binom{n}{k}$ is the number of ways to choose k objects from n distinct objects. Examples: $\binom{4}{3} = 4$. Why? Because there are 4 ways to choose 3 objects from 4 distinct objects: $\{1, 2, 3\}$, $\{1, 2, 4\}$, $\{1, 3, 4\}$, $\{2, 3, 4\}$.
 - $\binom{n}{k} = \frac{n!}{k!(n-k)!}$. Examples: $\binom{4}{3} = \frac{4!}{3!1!} = \frac{24}{6} = 4$.
- **Permutations:** $P(n, k)$ is the number of ways to choose k objects from n distinct objects and arrange them in a line. Examples: $P(4, 2) = 12$. Why? Because there are 12 ways to choose 2 objects from 4 distinct objects and arrange them in a line: $\{1, 2\}$, $\{1, 3\}$, $\{1, 4\}$, $\{2, 1\}$, $\{2, 3\}$, $\{2, 4\}$, $\{3, 1\}$, $\{3, 2\}$, $\{3, 4\}$, $\{4, 1\}$, $\{4, 2\}$, $\{4, 3\}$.
 - $P(n, k) = \frac{n!}{(n-k)!}$. Examples: $P(4, 2) = \frac{4!}{2!} = \frac{24}{2} = 12$.
 - So order matters in permutations but not in combinations. For example, $\{1, 2\}$ and $\{2, 1\}$ are different permutations but the same combination.

//Stringstream

```
string s = " I am Imran. I am 25 years old. ";  
stringstream ss(s); // ss is now a stream that contains the string s  
cout << ss.str() << '\n'; // print the string in the stream  
string word;  
while (ss >> word) { // read words from the stream, almost like cin >> word  
    cout << word << '\n';  
}  
// So this is useful when you want to get all the words from a string one by one
```

//Dijkstra with priority queue

```
#include <bits/stdc++.h>
using namespace std;
#define endl "\n"
#define ll unsigned long long int
void Dijkstra( vector<vector<pair<int,int>>>&graph, vector<pair<int,int>>&table, int start, int
vertices )
{
    for( int i = 1; i <= vertices; i++ )
    {
        if( i == start )
        {
            table[i] = make_pair(0,i);
        }
        else
        {
            table[i] = make_pair(INT_MAX,NULL);
        }
    }
    int total_visited = 0;
    vector<int>visited(vertices+1);
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>>Queue;
    Queue.push(make_pair(table[start].first,start));
    while( !Queue.empty() )
    {
        pair<int,int>current = Queue.top();
        Queue.pop();
        int vertex, distance;
        vertex = current.second;
        distance = current.first;
        if( visited[vertex] == 1 )
        {
            continue;
        }
        visited[vertex] = 1;
        total_visited++;
        for( auto point : graph[vertex] )
        {
            int node, weight, previous_distance, new_distance;
            node = point.first;
            if( visited[node] == 1 )
            {
                continue;
            }
            weight = point.second;
```

```
        previous_distance = table[node].first;
        new_distance = table[vertex].first+weight;
        if( new_distance < previous_distance )
        {
            table[node].first = new_distance;
            table[node].second = vertex;
            Queue.push(make_pair(new_distance,node));
        }
    }
}
}
int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int vertex, edge;
    cin >> vertex >> edge;
    vector<vector<pair<int,int>>>Graph(vertex+1);
    for( int i = 0; i < edge; i++ )
    {
        int v1, v2, weight;
        cin >> v1 >> v2 >> weight;
        Graph[v1].push_back(make_pair(v2,weight));
        Graph[v2].push_back(make_pair(v1,weight));
    }
    vector<pair<int,int>>Table(vertex+1);
    Dijkstra(Graph,Table,1,vertex);
    if( Table[vertex].first == INT_MAX )
    {
        cout << -1 << endl;
    }
    else
    {
        int current = vertex;
        vector<int>Path;
        Path.push_back(current);
        while( current != 1 )
        {
            current = Table[current].second;
            Path.push_back(current);
        }
        for( auto itr = Path.rbegin(); itr != Path.rend(); itr++ )
        {
            cout << *itr << " ";
        }
    }
}
```

//Total number of divisors of all numbers from 1 to 1000001

```
void Func( vector<int>&divisors, int n )
{
    for( int i = 1; i <= n; i++ )
    {
        for( int j = i; j <= n; j += i )
        {
            divisors[j]++;
        }
    }
}
```

//Difference Array

```
#include <bits/stdc++.h>
using namespace std;
#define endl "\n"
#define printDA for( auto point : DA ){cout << point << " " << endl;}
void Difference_Array( vector<int>&DA, vector<int>arr )
{
    int length = arr.size();
    DA.push_back(arr[0]);
    for( int i = 1; i < length; i++ )
    {
        DA.push_back(arr[i]-arr[i-1]);
    }
}
void Update_DA( vector<int>&DA, int l, int r, int value )
{
    DA[l] += value;
    if( r < DA.size()-1 )
    {
        DA[r+1] -= value;
    }
}
void Print_DA( vector<int>&DA, vector<int>&A )
{
    int length = DA.size();
    cout << "Print " << length << endl;
    for( int i = 0; i < length; i++ )
    {
        if( i == 0 )
        {
            A.push_back(DA[i]);
        }
        else
        {
            A.push_back(DA[i]+A[i-1]);
        }
    }
}
```

```
    }
    cout << A[i] << " ";
}
}
int main()
{
    int length;
    cin >> length;
    vector<int>store;
    for( int i = 0; i < length; i++ )
    {
        int value;
        cin >> value;
        store.push_back(value);
    }
    vector<int>DA;
    Difference_Array(DA,store);
    printDA;
    Update_DA(DA,2,4,10);
    vector<int>Array;
    Print_DA(DA,Array);
}
```

//DSU

```
class DSU{
private:
    int Total_Union;
    vector<int>Parent, Union_Size;
public:
    DSU( int vertices )
    {
        Total_Union = vertices;
        for( int i = 0; i <= vertices; i++ )
        {
            Parent.push_back(i);
            Union_Size.push_back(1);
        }
    }
    void Union( int vertex1, int vertex2 )
    {
        int union1, union2;
        union1 = Find_Union(vertex1);
        union2 = Find_Union(vertex2);
        if( union1 != union2 )
        {
            int first, second;
            if( Union_Size[union1] >= Union_Size[union2] )
            {
                first = union1;
                second = union2;
            }
            else
            {
                first = union2;
                second = union1;
            }
            Parent[second] = first;
            Union_Size[first] += Union_Size[second];
            Union_Size[second] = 0;
            Total_Union--;
        }
    }
    int Find_Union( int vertex )
    {
        int current = vertex;
        while( current != Parent[current] )
```

```
    {
        current = Parent[current];
    }
    int root = current;
    while( Parent[vertex] != root )
    {
        int temp = Parent[vertex];
        Parent[vertex] = root;
        vertex = temp;
    }
    return root;
}
int Unions()
{
    return Total_Union;
}
};
```

//Concept of the distance of the furthest node from any node

```
void Depth_Node( vector<vector<int>>&tree, vector<int>&level, int &Max_Depth, int
&Deepest_Node, int vertices, int start )
{
    vector<int>visited(vertices+1);
    stack<int>Stack;
    Stack.push(start);
    visited[start] = 1;
    level[start] = 0;
    while( !Stack.empty() )
    {
        int node = Stack.top();
        Stack.pop();
        if( level[node] >= Max_Depth )
        {
            Max_Depth = level[node];
            Deepest_Node = node;
        }
        for( auto point : tree[node] )
        {
            if( visited[point] != 1 )
            {
                visited[point] = 1;
                Stack.push(point);
                level[point] = level[node]+1;
            }
        }
    }
}

int main()
{
    int n, start = 0;
    cin >> n;
    vector<vector<int>>Tree(n+1);
    for( int i = 0; i < n-1; i++ )
    {
        int vertex1, vertex2;
        cin >> vertex1 >> vertex2;
        if( start == 0 )
        {
            start = vertex1;
        }
        Tree[vertex1].push_back(vertex2);
        Tree[vertex2].push_back(vertex1);
    }
    int Max_Depth = 0, Deepest_Node;
```

```
vector<int>Level_One(n+1,0), Level_Two(n+1,0);
Depth_Node(Tree,Level_One,Max_Depth,Deepest_Node,n,start);
int first = Deepest_Node;
Depth_Node(Tree,Level_Two,Max_Depth,Deepest_Node,n,first);
int second = Deepest_Node;
vector<int>first_distance(n+1), second_distance(n+1);
Depth_Node(Tree,first_distance,Max_Depth,Deepest_Node,n,first);
Depth_Node(Tree,second_distance,Max_Depth,Deepest_Node,n,second);
for( int i = 1; i <= n; i++ )
{
    cout << max(first_distance[i],second_distance[i]) << " ";
}
cout << "\n";
}
```

Connected Components In A Graph

```
#include <bits/stdc++.h>
using namespace std;
int connected_components( vector<vector<int>>&graph, int vertices )
{
    int total = 0, unvisited = vertices;

    unordered_map<int,int>visited;

    while( unvisited != 0 )
    {
        total++;
        int start;
        for( int i = 1; i <= vertices; i++ )
        {
            if( visited[i] != 1 )
            {
                start = i;
                break;
            }
        }
        stack<int>Stack;
        Stack.push(start);
        visited[start] = 1;
        unvisited--;
        while( !Stack.empty() )
        {
            int vertex = Stack.top();
            Stack.pop();
            for( int adjacent : graph[vertex] )
            {
                if( visited[adjacent] != 1 )
                {
                    Stack.push(adjacent);
                    visited[adjacent] = 1;
                    unvisited--;
                }
            }
        }
    }
    return total;
}

int main()
{
    int nodes, edges;

    cin >> nodes >> edges;
    vector<vector<int>>Graph(nodes+1);
    for( int i = 0; i < edges; i++ )
    {
        int vertex1, vertex2;
        cin >> vertex1 >> vertex2;
        Graph[vertex1].push_back(vertex2);
        Graph[vertex2].push_back(vertex1);
    }
    cout << connected_components(Graph, nodes) << endl;
}
```

Minimum Spanning Tree Cost

```
int MST_Cost( vector<vector<pair<int,int>>>&graph, int vertices, int start )
{
    vector<int>visited(vertices,0);
    int target_edge = vertices-1, found_edge = 0, cost = 0;
    priority_queue< pair<int,pair<int,int>>, vector<pair<int,pair<int,int>>>,
greater<pair<int,pair<int,int>>> >Queue;
    for( auto point : graph[start] )
    {
        Queue.push(make_pair(point.second,make_pair(start,point.first)));
    }
    visited[start] = 1;
    while( !Queue.empty() && found_edge != target_edge )
    {
        pair<int,pair<int,int>>current;
        current = Queue.top();
        Queue.pop();
        int vertex1, vertex2, weight;
        weight = current.first;
        vertex1 = current.second.first;
        vertex2 = current.second.second;
        if( visited[vertex1] == 1 && visited[vertex2] == 1 )
        {
            continue;
        }
        cost += weight;
        int New;
        if( visited[vertex1] != 1 )
        {
            New = vertex1;
        }
        else
        {
            New = vertex2;
        }
        visited[New] = 1;
        for( auto point : graph[New] )
        {
            if( visited[point.first] != 1 )
            {
                Queue.push(make_pair(point.second,make_pair(New,point.first)));
            }
        }
    }
    return cost;
}

parent[point] = node;
```

```

    }
    else if( color[point] == 1 )
    {
        int ancestor = parent[point], current = node;
        vector<int>store;
        while( current != ancestor )
        {
            store.push_back(current);

            current = parent[current];
        }
        store.push_back(ancestor);
        store.push_back(point);
        cycle.push_back(store);
    }
}
}
}
}
}

```

```

int main()
{
    int n, m;
    cin >> n >> m;
    vector<vector<int>>>Graph(n+1), cycle;
    for( int i = 0; i < m; i++ )
    {
        int vertex1, vertex2;
        cin >> vertex1 >> vertex2;
        Graph[vertex1].push_back(vertex2);
        Graph[vertex2].push_back(vertex1);
    }
    Detect_Cycle(Graph, cycle, n);
    int total_cycle = cycle.size();

    cout << "Total Cycle: " << total_cycle << endl;
    for( int i = 0; i < total_cycle; i++ )
    {
        cout << "Cycle " << i+1 << ": ";
        for( auto point : cycle[i] )
        {
            cout << point << " ";
        }
        cout << endl;
    }
}

```

Detect all cycles from all connected components in a graph

```

#include <bits/stdc++.h>
using namespace std;
void Detect_Cycle( vector<vector<int>>&graph, vector<vector<int>>&cycles, vector<int>&found,
vector<int>&color, vector<int>&Parent, int vertex, int parent )
{
    found[vertex] = 1;
    if ( color[vertex] == 2 )
    {
        return;
    }
    if ( color[vertex] == 1 )
    {
        vector<int>store;
        int current = parent;
        store.push_back(current);
        while ( current != vertex )
        {
            current = Parent[current];
            store.push_back(current);
        }
        cycles.push_back(store);
    }
}

```

```
        return;
    }
    Parent[vertex] = parent;
    color[vertex] = 1;
    for ( auto point : graph[vertex])
    {
        if ( point == Parent[vertex] )
        {
            continue;
        }
        Detect_Cycle(graph,cycles,found,color,Parent,point,vertex);
    }
    color[vertex] = 2;
}

int main()
{
    int n, m;
    cin >> n >> m;
    vector<vector<int>>>Graph(n+1), Cycles;
    for( int i = 0; i < m; i++ )
    {
        int vertex1, vertex2;
        cin >> vertex1 >> vertex2;
        Graph[vertex1].push_back(vertex2);
        Graph[vertex2].push_back(vertex1);
    }
    vector<int>color(n+1), Parent(n+1), Found(n+1);
    for( int i = 1; i <= n; i++ )
    {
        if( Found[i] != 1 )
        {
            Detect_Cycle(Graph,Cycles,Found,color,Parent,i,i);
        }
    }
    int total_cycle = Cycles.size();
    cout << "Total_Cycle: " << total_cycle << endl;
    for( int i = 0; i < total_cycle; i++ )
    {
        cout << "Cycle " << i+1 << ": ";
        for( auto point : Cycles[i] )
        {
            cout << point << " ";
        }
        cout << endl;
    }
}
```

Fast input output

```
ios_base::sync_with_stdio(false);  
cin.tie(NULL);
```

ULLONG_MAX (Maximum value for a variable of type unsigned long long (All positive))

LLONG_MIN

LLONG_MAX

For sorting `vector<pair<int,int>>Pair;`

```
sort(vect.begin(), vect.end(), sortbysec);
```

```
sort(Pair.rbegin(),Pair.rend());
```

//Counting childs of every parent node

```
#include <bits/stdc++.h>  
using namespace std;  
int Recursive_DFS( vector<vector<int>>&tree, vector<int>&hashmap, int root )  
{  
    int leaf = 0;  
    for( auto point : tree[root] )  
    {  
        leaf += Recursive_DFS(tree,hashmap,point);  
        leaf++;  
    }  
    hashmap[root] = leaf;  
    return leaf;  
}  
int main()  
{  
    ios_base::sync_with_stdio(false);  
    cin.tie(NULL);  
    int n;  
    cin >> n;  
    vector<vector<int>>Tree(n+1);  
    vector<int>Hashmap(n+1);  
    for( int i = 2; i <= n; i++ )  
    {  
        int value;  
        cin >> value;  
        Tree[value].push_back(i);  
    }  
    Recursive_DFS(Tree,Hashmap,1);  
    for( int i = 1; i <= n; i++ )  
    {  
        cout << Hashmap[i] << " ";  
    }  
    cout << "\n";  
}
```

The following function prime checks if the given number n is prime. $O(\sqrt{n})$ complexity

```
bool prime(int n) {  
    if (n < 2) return false;  
    for (int x = 2; x*x <= n; x++) {  
        if (n%x == 0) return false;  
    }  
    return true;  
}
```

Distance Between Two Points

Distance between (x_1, y_1) and (x_2, y_2) :

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Used in geometry problems to calculate the Euclidean distance.

//XOR

- $a \oplus a = 0$
- $a \oplus 0 = a$
- $a \oplus b = 0 \implies a = b$
- $a \oplus b = b \oplus a$
- $(a \oplus b) \oplus c = a \oplus (b \oplus c)$
- $a \oplus b \oplus a = (a \oplus a) \oplus b = 0 \oplus b = b$
- $a \oplus a \oplus \dots \oplus a = 0$ (even number of a 's)
- $a \oplus a \oplus \dots \oplus a = a$ (odd number of a 's)
- $a \oplus b = c \implies a = b \oplus c \implies a \oplus b \oplus c = 0$
- **The XOR Trick:** If we have a sequence of XOR operations $a \oplus b \oplus c \dots$ then we can remove all pairs of duplicated values without affecting the result.
- **Missing Number:** Given an array of $n - 1$ distinct integers in the range $[1, n]$. Find the missing number in $O(n)$ time.
- **Duplicate Number:** Given an array of $n + 1$ integers in the range $[1, n]$ where exactly one element appears twice. Find the duplicate number in $O(n)$ time.

//Precision

```
double num = 103.1415926;
```

```
cout << fixed << setprecision(3) << num << '\n'; // 103.142
```

```
cout << fixed << setprecision(5) << num << '\n'; // 103.14159
```

//Bits et

A bitset is a container that stores bits (elements with only two possible values: 0 or 1, true or false, ...). The class emulates an array of bool elements, but optimized for space allocation: generally, each element occupies only one bit.

- Declaration: `bitset<size> bitset_name;` Example: `bitset<10> bs;` Note that, the size must be a constant and it can't be changed later.
- Initialization:
 - `bitset_name = bitset<size>(string);` Example: `bs = bitset<10>("1010101011");`
 - `bitset_name = bitset<size>(unsigned long long);` Example: `bs = bitset<10>(1001);` The binary representation of 1001 is 1111101001. So this will set the bitset to be = 1111101001.
 - `bitset<10> bs;` This will create a bitset with all bits set to 0.
- Accessing elements: `bitset_name[i]` to access the i-th bit. Note that, the least significant bit is at index 0. So `bs[0]` will return the rightmost bit.
- Printing a bitset: `cout << bitset_name;` Example: `cout << bs;` This will print the bitset from left to right. So `cout << bs;` from above will print 1111101001.
- It supports all the bitwise operators like `&`, `|`, `^`, `<<`, `>>`, `~`, etc.

Assuming a fixed bitset `bs = 1111101001` and `p = 0110110100` as input for each function: (Let the size of the bitset be n and $W = 64$ for 64-bit computers and $W = 32$ for 32-bit computers)

FUNCTION	DEFINITION	SYNTAX	OUTPUT (MSB TO LSB)
<code>[]</code>	Access an element	<code>bs[1];</code>	<code>0</code>
<code>set</code>	Set a specific bit to 1	<code>bs.set(1);</code>	bs: 1111101011
<code>reset</code>	Reset a specific bit to 0	<code>bs.reset(0);</code>	bs: 1111101000
<code>flip</code>	Flip a specific bit	<code>bs.flip(0);</code>	bs: 1111101000
<code>set</code>	Set all bits to 1	<code>bs.set();</code>	bs: 1111111111
<code>reset</code>	Reset all bits to 0	<code>bs.reset();</code>	bs: 0000000000
<code>flip</code>	Flip all bits	<code>bs.flip();</code>	bs: 0000010110
<code>count</code>	Count the number of 1s	<code>bs.count();</code>	<code>7</code>
<code>size</code>	Get the size of the bs	<code>bs.size();</code>	<code>10</code>
<code>to_ulong</code>	Convert to unsigned long long	<code>bs.to_ulong();</code>	<code>1001</code>
<code>to_string</code>	Convert to string	<code>bs.to_string();</code>	<code>1111101001</code>
<code>any</code>	Check if any bit is set	<code>bs.any();</code>	<code>1</code> (true)
<code>none</code>	Check if no bit is set	<code>bs.none();</code>	<code>0</code> (false)
<code>all</code>	Check if all bits are set	<code>bs.all();</code>	<code>0</code> (false)
<code><<</code>	Left shift	<code>bs << 1;</code>	bs: 1111010010
<code>>></code>	Right shift	<code>bs >> 1;</code>	bs: 0111110100
<code>&</code>	Bitwise AND	<code>bs & p;</code>	bs: 0110100000
OR operator	Bitwise OR	similar	bs: 1111111101
<code>^</code>	Bitwise XOR	<code>bs ^ p;</code>	bs: 1001011101
<code>~</code>	Bitwise NOT	<code>~bs;</code>	bs: 0000010110

// Given a sorted array a of n integers where each element is either 0 or 1. Find the index of the first 1 in the array. If there is no 1 present in the array, output -1.

```
#include<bits/stdc++.h>

using namespace std;

int32_t main() {
    ios_base :: sync_with_stdio(0);
    cin.tie(0);

    int n; cin >> n;

    int a[n + 1];
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    int l = 1, r = n, ans = -1;

    while (l <= r) { // run while there is at least one element in the search interval
        int mid = (l + r) / 2;

        if (a[mid] == 1) {
            ans = mid;
            r = mid - 1;
        }
        else {
            l = mid + 1;
        }

        // do not set l = mid or r = mid
        // always set mid + 1 or mid - 1
        // what happens if you set l = mid or r = mid while doing while (l <= r) ?
        // try with l = 1 and r = 1 and you will find out
    }

    cout << ans << '\n';

    return 0;
}
```

//A function is monotonic if it is either entirely nonincreasing or nondecreasing. Let's say we want to find the maximum or minimum value for which a certain condition is true and the condition is monotonic, that is, if the condition is true for a certain value, it will be true for all values greater or less than that value.

```
#include<bits/stdc++.h>

using namespace std;

const int N = 1e5 + 9;

int a[N], n;

long long s;

bool f(int x) {

    long long sum = 0;

    for (int i = 1; i <= n; i++) {

        sum += a[i] / x;

        return sum >= s;

    }

    int32_t main() {

        ios_base :: sync_with_stdio(0);

        cin.tie(0);
```

```
cin >> n >> s;

for (int i = 1; i <= n; i++) {

    cin >> a[i];

    int l = 1, r = 1e9, ans = -1;

    while (l <= r) {

        int mid = (l + r) / 2;

        if (f(mid)) {

            ans = mid;

            l = mid + 1;

        }

        else {

            r = mid - 1;

        }

    }

    cout << ans << '\n';

    return 0;
```

